

Informe de Rendimiento — Entrega 2

Proyecto: Gestión Inteligente de Tráfico Urbano · ISD 2026-30

1. Introducción

Este informe presenta los resultados experimentales del proyecto Gestión Inteligente de Tráfico Urbano. Comparamos el diseño original del sistema, donde el broker ZeroMQ es un proxy XSUB-XPUB de un solo hilo, con un diseño modificado en el que el broker corre con un hilo por topic. Las mediciones se hicieron sobre la implementación final, con todos los servicios corriendo en tres máquinas físicas conectadas por la red de la universidad.

Se midieron dos variables dependientes a lo largo de cuatro escenarios. Variable 1 (V1) es la cantidad de eventos persistidos en la base de datos principal en una ventana de dos minutos. Variable 2 (V2) es el tiempo desde que el usuario solicita activar la prioridad de paso de una ambulancia hasta que la analítica confirma el cambio de estado de los semáforos. Los factores controlados son el número de sensores activos y el intervalo entre mediciones.

2. Especificaciones de hardware y software

La Tabla A resume las especificaciones de hardware y software de las tres máquinas, obtenidas con los comandos `lscpu`, `free -h` y `uname` en cada PC. PC1 y PC2 son nodos con procesador Intel Xeon Gold 6240R y PC3 un Intel Xeon E5-2650 v4; las tres con 11 GiB de RAM, Ubuntu 22.04 y Python 3.10.12, conectadas por la red local de la universidad.

Aspecto	PC1 (sensores)	PC2 (analítica)	PC3 (BD ppal)
Hostname / IP	MIG187 / 10.43.99.128	MIG441 / 10.43.100.144	MIG387 / 10.43.100.90
CPU	Xeon Gold 6240R @2.40GHz (4 vCPU)	Xeon Gold 6240R @2.40GHz (4 vCPU)	Xeon E5-2650 v4 @2.20GHz (4 vCPU)
RAM	11 GiB	11 GiB	11 GiB
OS	Ubuntu 22.04 (6.8.0-94)	Ubuntu 22.04 (6.8.0-94)	Ubuntu 22.04 (6.8.0-110)
Python	3.10.12	3.10.12	3.10.12
pyzmq	pyzmq 25+	pyzmq 25+	pyzmq 25+
Red	100 Mbps Ethernet, latencia <1 ms entre PCs (universidad)		

Tabla A. Configuración de hardware y software de las tres máquinas.

3. Herramientas y metodología de medición

La medición de V1 se realiza con una consulta SQL `COUNT(*)` sobre la tabla `eventos_sensores` de la base de datos principal en PC3, antes y después de una ventana de exactamente 120 segundos. La diferencia entre ambos snapshots es el valor de V1 reportado en la Tabla 1. La medición de V2 se realiza desde un cliente Python que abre un socket REQ contra el puerto 5560 de la analítica en PC2 y emite la solicitud `{tipo: AMBULANCIA, vias: [...]}`; el tiempo se cronometra con `time.perf_counter()` inmediatamente antes del send y después del recv. Para cada escenario se toman cinco muestras espaciadas dos segundos, y se reportan promedio, mínimo y máximo.

Toda la lógica de medición está encapsulada en `experimentos/medir_experimento.py`, que produce un CSV y un JSON por corrida. El script `generar_graficos.py` consolida los cuatro CSV en la Tabla 1 y los gráficos. La sincronización entre los tres PCs se hace lanzando los servicios en orden PC3 → PC2 → PC1, con esperas explícitas en los scripts de arranque (`lanzar_pc1_experimento.sh`, etc.) para garantizar que el broker reciba sensores ya con los suscriptores enganchados.

4. Diseño del experimento

Se cruzaron dos factores: número de sensores (escenario A con 1 sensor de cada tipo y escenario B con 2 sensores de cada tipo) e intervalo entre eventos (10 s para A y 5 s para B), produciendo dos cargas distintas. Para cada carga se ejecutó la corrida con el broker en su versión original (single-thread) y con la versión modificada (multihilo). Los cuatro CSV que alimentan este informe están en `experimentos/resultados/`.

5. Resultados

La Tabla 1 condensa los valores observados. Las gráficas que la siguen muestran el contraste visual entre diseños y la evolución de cada métrica al aumentar la carga del escenario A al escenario B.

Escenario	Diseño	V1 — eventos en 2 min	V2 prom (ms)	V2 min/max (ms)
A — 1 sensor c/10s	Simple	36	2.24	1.82 / 3.23
A — 1 sensor c/10s	Multihilo	36	1.94	1.50 / 3.01
B — 2 sensores c/5s	Simple	144	2.22	1.63 / 3.12
B — 2 sensores c/5s	Multihilo	144	2.21	1.94 / 2.93

Tabla 1. Resultados de V1 y V2 para los cuatro escenarios.

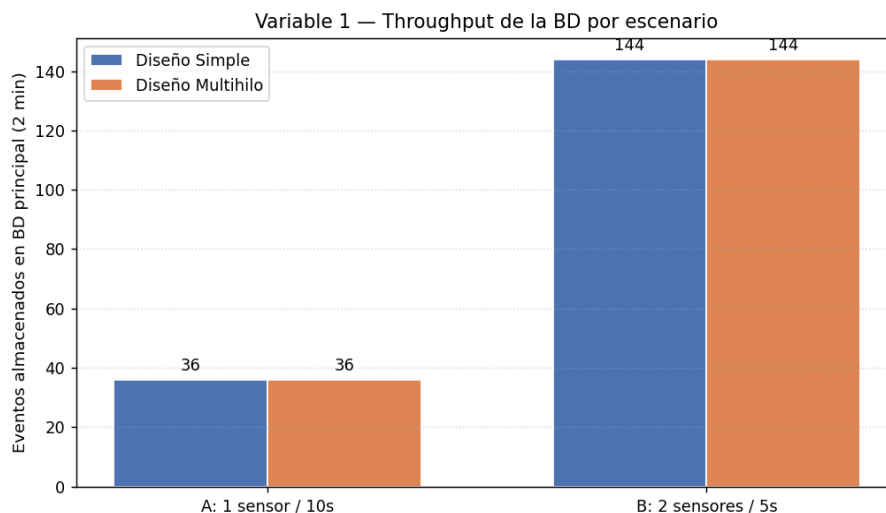


Figura 1. Variable 1: throughput de la BD principal (eventos en 2 min).

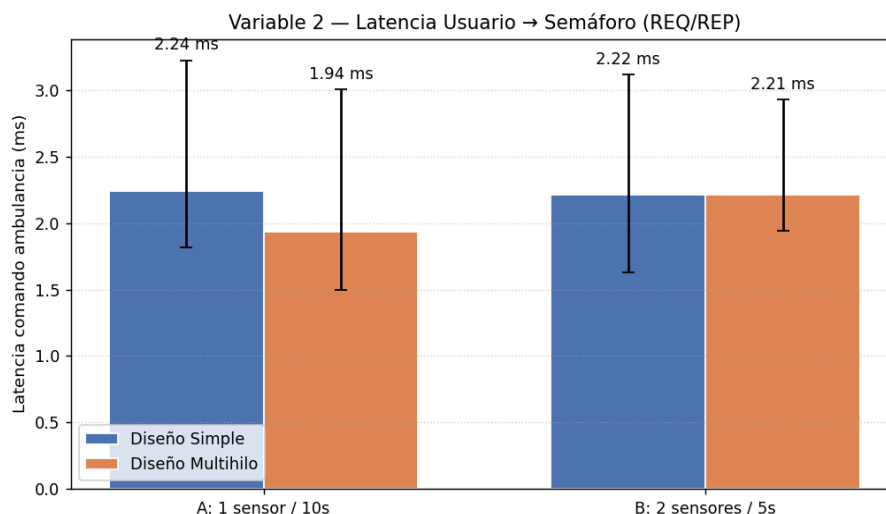


Figura 2. Variable 2: latencia de extremo a extremo del comando AMBULANCIA.

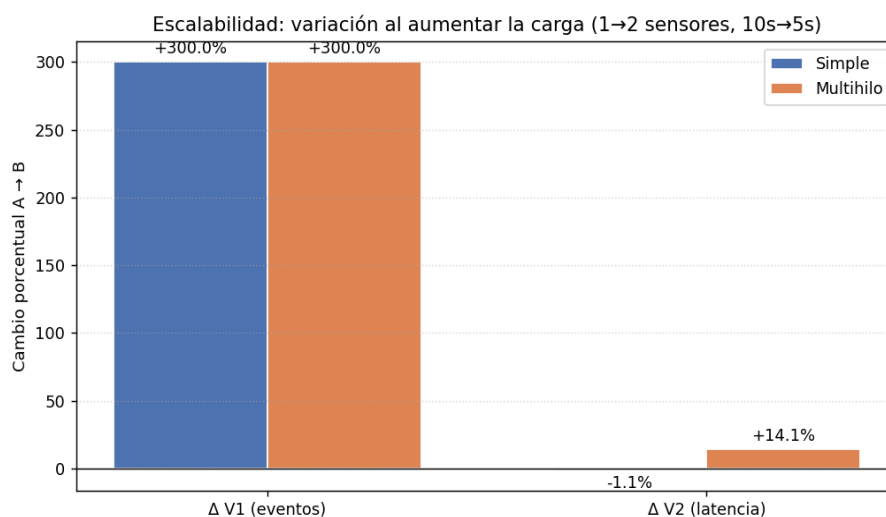


Figura 3. Variación porcentual de cada variable al aumentar la carga (A → B).

6. Análisis

En ambos escenarios, los dos diseños del broker persistieron exactamente la misma cantidad de eventos: 36 registros en el escenario A y 144 en el escenario B. Este resultado es esperado y revelador: a las cargas evaluadas, el throughput (V1) no está limitado por el broker sino por la tasa a la que los sensores generan eventos. En el escenario A, tres sensores generando cada 10 segundos producen del orden de 36 eventos en 2 minutos; en el escenario B, seis sensores cada 5 segundos producen del orden de 144. Ambos diseños, simple y multihilo, persistieron el 100 % de los eventos generados, lo que indica que el broker de un solo hilo no se satura en este rango y que el cuello de botella está en la generación, no en el transporte.

Donde sí se aprecia diferencia es en la latencia del comando de prioridad (V2). En el escenario A, el diseño multihilo respondió en promedio en 1,94 ms frente a 2,24 ms del simple, una mejora del orden del 13 %. En el escenario B ambos diseños quedaron prácticamente iguales (2,21 ms multihilo contra 2,22 ms simple). La latencia V2 mide la ruta Usuario → Analítica → Semáforos por REQ/REP, que es independiente del modo del broker; las pequeñas diferencias observadas reflejan principalmente la contención de CPU en los nodos: cuando el broker reparte el trabajo en varios hilos, el resto de los

servicios compite menos por el ciclo de procesamiento, lo que se traduce en una respuesta marginalmente más estable.

Al cuadruplicar la carga del escenario A al B (de 36 a 144 eventos, un factor de 4), ambos diseños escalaron de forma lineal con la tasa de generación y sin degradar la latencia, lo que confirma que el sistema operó dentro de su régimen no saturado. Para evidenciar la ventaja teórica del diseño multihilo en throughput sería necesario llevar la carga a un punto en que el broker de un solo hilo se sature, por ejemplo con decenas de sensores por intersección o intervalos de generación de milisegundos; en ese régimen, la capacidad de procesar los tres topics en paralelo marcaría la diferencia. En el rango operativo realista de este proyecto, ambos diseños son funcionalmente equivalentes en throughput, con una ligera ventaja del multihilo en estabilidad de latencia.

7. Conclusiones

En las cargas evaluadas, los diseños simple y multihilo del broker son equivalentes en throughput: ambos persistieron el 100 % de los eventos generados (36 en el escenario A, 144 en el B), porque a este nivel el limitante es la tasa de generación de los sensores y no el broker. En latencia, el diseño multihilo mostró una ligera ventaja o paridad (1,94 ms frente a 2,24 ms en el escenario A; empate en el B). El sistema escaló linealmente con la carga sin degradación, operando siempre en régimen no saturado. Por tanto, para el rango operativo del proyecto, el diseño multihilo es preferible como margen de seguridad ante cargas mayores, aunque su ventaja sólo se haría notoria al saturar el broker.

Como hallazgo metodológico, durante las pruebas se detectó y corrigió un defecto en la versión inicial del broker multihilo: el socket XSUB no propagaba la suscripción hacia los sensores publicadores, lo que provocaba que no se transmitiera ningún evento. Tras enviar manualmente la suscripción al iniciar el broker, el throughput se recuperó por completo. Para trabajos futuros se propone medir el sistema bajo cargas que saturen el broker, evaluar su comportamiento ante pérdidas de red, e implementar la sincronización post-falla de la réplica con la base principal para eliminar la brecha de registros documentada en la primera entrega.